Universidad de Murcia

Facultad de Matemáticas

Grado en Matemáticas
Tecnología de la Programación

Proyecto TDA

Realizado por:

Francisco Carlos Bernad Guillén
Francisco Pellicer Alarcón

Curso 2024-2025

Índice

1. TDA Correo
1.1 Especificación ..... (página 3)
1.1.1 Nombre y descripción …….(página 3)
1.1.2 Dominio de los datos……(página 3)
1.1.3 Especificación de las operaciones……(página 3)
1.2 Representación .....……...(página 5)
1.3 Implementación ..... (página 5)
1.3.1 Estructura del TDA ..... (página 6)
1.3.2 Justificación ..... (página 7)
2. TDA Gestor de Correo
2.1 Especificación ..... (página 8)
2.1.1 Nombre y descripción ……(página 8)
2.1.2 Dominio de los datos……..(página 8)
2.1.3 Especificación de las operaciones……..(página 9)
2.2 Representación ..... (página 13)
2.3 Implementación ..... (página 14)
2.3.1 Estructura del TDA .....(página 14)
2.3.2 Justificación ..... (página 17)
3. TDA DataFrame
3.1 Especificación .....(página 18)
3.1.1 Nombre y descripción ……(página 18)
3.1.2 Dominio de los datos……..(página 18)
3.1.3 Especificación de las operaciones……..(página 18)
3.2 Representación ..... (página 23)
3.3 Implementación ..... (página 23)
3.3.1 Estructura del TDA ..... (página 23)
3.3.2 Justificación ..... (página 25)
4. Programa Principal ..... (página 26)

TDA Correo

Francisco Carlos Bernad Guillén y Francisco Pellicer Alarcón

December 9

Nombre y Descripción

Nombre: Correo
Descripción:
El TDA Correo representa un correo electrónico con campos como remitente, destinatario, asunto, mensaje y fechas de envío y recepción. Permite gestionar el estado del correo (borrador o enviado), modificar campos, y buscar palabras en el contenido. Además, maneja excepciones, como campos incompletos o intentos de modificar correos ya enviados.

Dentro de la clase, la subclase `Fecha` gestiona las fechas del correo y permite generar fechas aleatorias dentro de ciertas restricciones.

Dominio de los Datos

El dominio de los datos de un correo incluye:

- **Fecha** (F)
- **Enteros** (Z)
- **Strings(Cadenas de texto)** (S)
- **Booleanos** (B)

Especificación de las Operaciones

1. `cambiar_estado_de_lectura(self) -> bool`
   - **Descripción**: Cambia el estado de lectura de un correo si este ha sido enviado.
   - **Precondiciones**:
     - El correo debe haber sido enviado previamente.
   - **Retorna**:
     - `True` si se cambió correctamente el estado de lectura.

- **Excepciones**:
  - *ErrorMensajeNoEnviado*: Si el correo no ha sido enviado.

2. `enviar_correo(self) -> bool`
   - **Descripción**: Envía un correo asegurándose de que todos los campos obligatorios estén completos.
   - **Precondiciones**:
     - Todos los campos obligatorios del correo deben estar completados.
   - **Retorna**:
     - `True` si el correo se envió correctamente.
   - **Excepciones**:
     - *ErrorCorreoYaEnviado*: Si el correo ya había sido enviado previamente.
     - *ErrorCamposNoCompletos*: Si faltan campos obligatorios por completar.

3. `modificar_campos(self, remitente: str = None, destinatario: str = None, asunto: str = None, cuerpo: str = None) -> bool`
   - **Descripción**: Permite modificar los campos de un correo que aún está en borrador.
   - **Precondiciones**:
     - El correo debe estar en estado de borrador.
     - Al menos uno de los parámetros debe ser diferente de `None`.
   - **Retorna**:
     - `True` si los campos se modificaron correctamente.
   - **Excepciones**:
     - *ErrorCorreoYaEnviado*: Si se intenta modificar un correo ya enviado.
     - *ErrorCamposAusentes*: Si no se proporciona ningún parámetro para modificar.

4. `buscar_palabra(self, palabra_a_buscar: str) -> bool`
   - **Descripción**: Busca una palabra específica dentro del asunto y el mensaje del correo.
   - **Precondiciones**:
     - `palabra_a_buscar` debe ser una cadena válida.
   - **Retorna**:
     - `True` si la palabra se encuentra en el asunto o el mensaje.

    - `False` en caso contrario.
  - **Excepciones**: Ninguna.

5. `__str__(self) -> str`
   - **Descripción**: Devuelve una representación en cadena del correo, mostrando su remitente, destinatario, asunto, mensaje y fechas de envío/recepción si aplica.
   - **Precondiciones**: Ninguna.
   - **Retorna**:
     - Una cadena con la representación detallada del correo.
   - **Excepciones**: Ninguna.

Representación Interna

La representación interna del TDA Correo es la siguiente:

```
struct Correo {
    string _remitente;
    string _destinatario;
    string _asunto;
    string _mensaje;
    Fecha _fecha_envio;
    Fecha _fecha_recepcion;
    bool _borrador;
    bool _leido;
    int _identificador;
}

struct Fecha {
    int _anyo;
    int _mes;
    int _dia;
    int _hora;
    int _minuto;
    int _segundo;
}
```

Implementación

El TDA `Correo` ha sido diseñado para modelar un correo electrónico con atributos esenciales como remitente, destinatario y cuerpo del correo. Además, cuenta con métodos para interactuar con estos campos y verificar su estado.

Estructura del TDA

La clase `Correo` incluye un atributo `_numero_correos`, que actúa como un contador para asignar un identificador único a cada correo, facilitando la comparación de correos por su identificador.

Subclases

La clase `Correo` contiene subclases internas que no se deben utilizar fuera del contexto de la clase principal, como:

- **Errores:** Subclases como `ErrorMensajeNoEnviado` y `ErrorCamposNoCompletos`, que manejan errores específicos al interactuar con el correo.
- **Subclase `Fecha`:** Gestiona las fechas de envío y recepción, generando fechas aleatorias dentro de las restricciones.

Métodos Getter y Setter

Los métodos `Getter` y `Setter` aseguran que los valores asignados a los atributos sean válidos, mejorando la integridad del TDA y facilitando futuras modificaciones sin afectar la lógica del código.

Métodos Auxiliares

Se implementan métodos auxiliares como:

- `_campos_obligatorios_rellenados`: Verifica que todos los campos necesarios estén completos.
- `_datos_por_rellenar`: Devuelve los campos faltantes.

Estos métodos mejoran la eficiencia y la legibilidad del código, y se mantienen privados para evitar el acceso directo.

Métodos Públicos

Los métodos públicos disponibles para el usuario permiten modificar, enviar y leer correos, sin exponer detalles internos:

- `cambiar_estado_de_lectura`
- `enviar_correo`
- `modificar_campos`
- `buscar_palabra`

Métodos mágicos

- `__str__`
- `__eq__`.

Métodos y Atributos Privados

Los métodos y atributos privados incluyen las subclases de errores y la subclase `Fecha`, y los métodos auxiliares explicados previamente. Estos son necesarios para mantener la integridad y la estructura interna del TDA.

¿Por qué Privados?

Estas implementaciones se mantienen privadas para proteger la consistencia del TDA, evitando accesos no controlados y facilitando modificaciones futuras sin afectar a los usuarios del TDA.

TDA Gestor de Correo

Francisco Carlos Bernad Guillén y Francisco Pellicer Alarcón

December 9

Nombre y Descripción

Nombre: Gestor de Correo

Descripción:

El TDA Gestor de Correo permite gestionar una colección de correos electrónicos. Cada correo electrónico contiene campos como remitente, destinatario, asunto, mensaje, fechas de envío y recepción, y estado de lectura. El Gestor de Correo permite realizar operaciones sobre una lista de correos, como enviar, modificar, buscar, y cambiar el estado de lectura de los correos. Además, se encarga de manejar excepciones relacionadas con correos no enviados, campos incompletos, y correos ya leídos.

Dentro de la clase, la subclase `Bag` tiene como propósito proporcionar una estructura para almacenar y gestionar correos electrónicos y criterios de spam, además de más funcionalidades sin preocuparse por el orden ni la duplicación de elemento

Dominio de los Datos

El dominio de los datos de un gestor de correos incluye:

- **Enteros** (Z)
- **Char(Caracteres)** (C)
- **Booleanos** (B)
- **Stings(Cadenas de texto** (S)
- **Bag** (Bg)
- **Correo** (C)
- **Valores nulos** (`None` o NaN):

Especificación de las Operaciones

Métodos Públicos

1. `len(self, bandeja: str) -> int`

   Descripción: Permite obtener el número de correos en la bandeja seleccionada.

   Precondiciones:

   - `bandeja` debe ser el nombre de una bandeja válida existente.

   Retorna:

   - `int`: El número de correos en la bandeja seleccionada.

   Excepciones:

   - Ninguna.

2. `mostrar_bandeja(self, bandeja_a_mostrar: str) -> str`

   Descripción: Muestra todos los correos de una bandeja o los criterios de spam.

   Precondiciones:

   - `bandeja_a_mostrar` debe ser el nombre de una bandeja válida.

   Retorna:

   - String con todos los correos o criterios de spam en la bandeja seleccionada.

   Excepciones:

   - Ninguna.

3. `cambiar_contrasenya(self, contrasenya: str) -> bool`

   Descripción: Cambia la contraseña del gestor de correo.

   Precondiciones:

   - `contrasenya` debe ser una cadena con al menos 8 caracteres.
   - La nueva contraseña no debe ser igual a la anterior.

   Retorna:

   - `True` si la contraseña se ha cambiado correctamente.

   Excepciones:

   - *GestorCorreo.ErrorContrasenyaInvalida*: Si la contraseña es demasiado corta o coincide con la anterior.

4. `anyadir_criterios_de_spam(self, criterio: str) -> bool`

   Descripción: Añade un nuevo criterio de spam si no existe previamente.

   Precondiciones:

   - `criterio` debe ser un string que se quiere añadir a los criterios de spam.

   Retorna:

   - `True` si el criterio se añadió correctamente.

   Excepciones:

   - *GestorCorreo.ErrorCriterioRepetido*: Si el criterio ya existe en la lista de spam.

5. `eliminar_criterios_de_spam(self, criterio: str) -> bool`

   Descripción: Elimina un criterio de spam si existe en la lista.

   Precondiciones:

   - `criterio` debe ser un string que se desea eliminar de los criterios de spam.

   Retorna:

   - `True` si el criterio se eliminó correctamente.

   Excepciones:

   - *GestorCorreo.ErrorCriterioNoEncontrado*: Si el criterio no se encuentra en la lista de spam.

6. `redactar_correo(self, destinatario: str = None, asunto: str = None, mensaje: str = None) -> Correo`

   Descripción: Crea un nuevo correo y lo almacena en la bandeja de borradores.

   Precondiciones:

   - `destinatario`, `asunto` y `mensaje` son opcionales, pero si se proporcionan, deben ser cadenas válidas.

   Retorna:

   - Instancia de `Correo` con los datos proporcionados.

   Excepciones:

   - Ninguna.

7. `modificar_correo(self, correo: Correo, destinatario: str = None, asunto: str = None, mensaje: str = None)`

   Descripción: Modifica los campos de un correo si este se encuentra en la bandeja de borradores.

   Precondiciones:

   - `correo` debe ser una instancia de `Correo`.
   - `destinatario`, `asunto` y `mensaje` son opcionales, pero si se proporcionan, deben ser cadenas válidas.

   Retorna:

   - `True` si el correo se modificó correctamente.

   Excepciones:

   - *GestorCorreo.ErrorCorreoNoEncontrado*: Si el correo no se encuentra en la bandeja de borradores.

8. `enviar_correo(self, correo: Correo, destinatario: "GestorCorreo")`

   Descripción: Envía un correo a un destinatario y lo clasifica según criterios de spam.

   Precondiciones:

   - `correo` debe ser una instancia de `Correo`.
   - `destinatario` debe ser una instancia válida de `GestorCorreo`.

   Retorna:

   - `True` si el correo fue enviado correctamente.

   Excepciones:

   - *GestorCorreo.ErrorCorreoNoEncontrado*: Si el correo no se encuentra en la bandeja de borradores.
   - *GestorCorreo.ErrorDestinatarioNoValido*: Si se intenta enviar un correo a un objeto que no es de tipo correo.

9. `cambiar_estado_lectura(self, correo: Correo)`

   Descripción: Cambia el estado de lectura de un correo si se encuentra en la bandeja de spam o borradores.

   Precondiciones:

   - `correo` debe ser una instancia de `Correo`.

   Retorna:

   - `True` si el estado de lectura fue cambiado correctamente.

Excepciones:

- *GestorCorreo.ErrorCorreoNoEncontrado*: Si el correo no se encuentra en la bandeja de spam o entrada.

10. `eliminar_correo(self, correo: Correo, bandeja_entrada: bool = None, bandeja_salida: bool = None, bandeja_borradores: bool = None, bandeja_spam: bool = None) -> bool`

    Descripción: Elimina un correo de una bandeja específica.

    Precondiciones:

    - `correo` debe ser una instancia de `Correo`.

    Retorna:

    - `True` si el correo fue eliminado correctamente.

    Excepciones:

    - *GestorCorreo.ErrorParametrosIncompletos*: Si no se especifica ninguna bandeja.
    - *GestorCorreo.ErrorCorreoNoEncontrado*: Si el correo no se encuentra en la bandeja indicada.

11. `busqueda_de_correos(self, cadena_texto: str, bandeja_entrada: bool = None, bandeja_salida: bool = None, bandeja_borradores: bool = None, bandeja_spam: bool = None) -> str`

    Descripción: Permite buscar correos que contengan una palabra o frase.

    Precondiciones:

    - `cadena_texto` debe ser una cadena no vacía.
    - Al menos una bandeja debe ser seleccionada para la búsqueda.

    Retorna:

    - String con los correos que contienen el texto buscado en las bandejas seleccionadas.

    Excepciones:

    - *GestorCorreo.ErrorBusquedaNoValida*: Si no se introduce ninguna bandeja o se intenta buscar palabras no válidas.

Representación Interna

La representación interna del TDA Gestor de Correo utiliza una estructura de lista enlazada. Se implementa a través de la clase `Bag`, que contiene nodos que almacenan correos. A continuación se muestra la estructura interna detallada:

```
struct GestorCorreo {
    Bag _bandeja_de_entrada;
    Bag _bandeja_de_salida;
    Bag _bandeja_de_borradores;
    Bag _bandeja_de_spam;
    Bag _criterios_de_spam;
}

struct Bag {
    Nodo _primer_nodo;
    int _longitud;
}

struct Nodo {
    Correo _elemento;
    Nodo _siguiente;
}

struct Correo {
    string _remitente;
    string _destinatario;
    string _asunto;
    string _mensaje;
    Fecha _fecha_envio;
    Fecha _fecha_recepcion;
    bool _borrador;
    bool _leido;
    int _identificador;
}

struct Fecha {
    int _anyo;
    int _mes;
    int _dia;
    int _hora;
    int _minuto;
    int _segundo;
}
```

En esta representación:

- `GestorCorreo` contiene varias instancias de la clase `Bag`, que son las bandejas de entrada, salida, borradores, spam, y los criterios de spam. Cada `Bag` gestiona los correos en forma de lista enlazada. - La clase `Bag` contiene nodos (`Nodo`), donde cada nodo almacena un correo (`Correo`) y una referencia al siguiente nodo en la lista. - Cada `Nodo` tiene dos atributos: `_elemento`, que es el correo almacenado, y `_siguiente`, que es una referencia al siguiente nodo de la lista. - La clase `Correo` representa los correos electrónicos con campos como remitente, destinatario, asunto, mensaje, fechas de envío y recepción, estado de borrador, estado de lectura y un identificador único.

Implementación del TDA Gestor de Correos

El **TDA Gestor de Correos** ha sido diseñado para modelar un sistema de gestión de correos electrónicos, permitiendo al usuario administrar las distintas bandejas disponibles y gestionar distintos criterios de spam. El gestor se apoya en una estructura auxiliar de tipo bolsa o `Bag` para el almacenamiento interno de los correos y utiliza excepciones personalizadas para manejar errores específicos.

Estructura Interna: `Bag`

La estructura interna `Bag` implementa una lista enlazada que almacena los correos y los criterios de spam. Se ha elegido este tipo de estructura ya que el orden en el que se almacenan los correos no es relevante. Esta implementación es eficiente para agregar y eliminar correos, por lo que resulta adecuada para gestionar bandejas dinámicas de correos.

La estructura `Bag` cuenta con la subclase `Nodo`, que es necesaria para la creación de una estructura enlazada. Algunos de los métodos que posee son:

- `add_item`
- `remove_item`
- `iterator`

Estos métodos permiten manipular la bolsa de manera eficiente.

Excepciones Personalizadas

Las demás subclases están diseñadas para manejar errores específicos, como `ErrorBandejaInexistente` o `ErrorCriterioRepetido`. Estas excepciones permiten manejar casos específicos de error con mensajes claros para el usuario, como en el caso de la clase `Correo`.

Atributos del Gestor

Cada objeto de la clase `Gestor` contiene atributos como:

- Nombre de usuario

- Contraseña (privada, no accesible directamente)
- Criterios de spam
- Cuatro bandejas de correos, cada una representada como una instancia independiente de tipo `Bag`, permitiendo que cada bandeja sea gestionada de manera aislada y personalizada.

El `Gestor` se puede instanciar indicando un usuario y una contraseña de al menos ocho caracteres. La contraseña es un atributo privado, lo que significa que no puede ser consultada, solo modificada. Al instanciar el gestor, se crean automáticamente las cinco bandejas.

Métodos Getter y Setter

El gestor cuenta con métodos Getter que controlan el acceso a los atributos de instancia de cada objeto. En este caso, cada bandeja tiene un método de acceso, pero solo se proporciona un método `set` para el atributo `contraseña`, ya que los demás atributos se gestionan desde la subclase `Bag`.

Métodos Auxiliares

Los siguientes métodos auxiliares son esenciales para mejorar la modularidad y reutilización del código:

- `_que_bandeja_es`: Este método devuelve la bandeja correspondiente, dependiendo del `String` introducido. Se ha implementado este método para centralizar la lógica de acceso a las bandejas y evitar repetir validaciones en distintos métodos.
- `_buscar_criterios_en_correo`: Método que verifica si el correo contiene alguna de las palabras definidas como criterios de spam. Se implementó para encapsular la lógica que busca criterios en un correo específico y permite reutilizar el código en distintos métodos.
- `_buscar_palabras_en_bandeja`: Método que busca correos dentro de una o varias bandejas y devuelve todos los correos encontrados con dichas palabras. Se introduce este método para mejorar la legibilidad del método `BuscarCorreos`.

Métodos Públicos

Los métodos públicos son aquellos que permiten al usuario interactuar directamente con el gestor de correos.

Gestión de Bandejas

- `mostrar_bandeja(bandeja_a_mostrar: str)`: Permite visualizar el contenido de una bandeja específica o los criterios de spam.
- `len(bandeja: str)`: Devuelve la cantidad de correos o elementos en una bandeja específica.

Gestión de Correos

- `redactar_correo(destinatario: str = None, asunto: str = None, mensaje: str = None)`: Permite crear un correo nuevo en la bandeja de borradores.
- `modificar_correo(correo: Correo, destinatario: str = None, asunto: str = None, mensaje: str = None)`: Permite modificar un correo en la bandeja de borradores.
- `enviar_correo(correo: Correo, destinatario: GestorCorreo)`: Envía un correo y lo clasifica automáticamente en bandejas según los criterios de spam.
- `cambiar_estado_lectura(correo: Correo)`: Cambia el estado de lectura de un correo en las bandejas de spam o entrada.

Gestión de Criterios de Spam

- `anyadir_criterios_de_spam(criterio: str)`: Añade un nuevo criterio de spam si no existe previamente.
- `eliminar_criterios_de_spam(criterio: str)`: Elimina un criterio de spam de la lista.

Búsquedas

- `busqueda_de_correos(cadena_texto: str, bandeja_entrada: bool = None, bandeja_salida: bool = None, bandeja_borradores: bool = None, bandeja_spam: bool = None)`: Realiza búsquedas avanzadas en las bandejas seleccionadas.

Gestión de Contraseñas

- `cambiar_contrasenya(contrasenya: str)`: Cambia la contraseña del gestor si cumple las validaciones.

Parte Privada

La parte privada incluye los métodos y atributos internos que gestionan la lógica del TDA y no están diseñados para ser utilizados directamente por el usuario. Estos elementos soportan las funcionalidades públicas, encapsulando los detalles técnicos.

Métodos Privados

Los métodos auxiliares mencionados anteriormente.

Atributos Privados

- `__contrasenya`: Contraseña del gestor.
  Justificación: Protege la seguridad del gestor al evitar accesos directos a la contraseña.

Justificación de la Implementación Pública y Privada

Métodos Públicos

Los métodos públicos permiten al usuario interactuar con el TDA de forma sencilla y segura. Se exponen solo las operaciones necesarias, evitando confusión o errores al usuario. Ejemplo: Métodos como `redactar_correo` encapsulan la lógica interna, ofreciendo una interfaz clara para la creación de correos.

Métodos Privados

Los métodos privados encapsulan la lógica compleja y las validaciones internas, asegurando que los datos y procesos se mantengan consistentes. Evitan accesos directos a elementos sensibles (como bandejas o contraseñas) que podrían causar inconsistencias si se modifican incorrectamente. Por ejemplo, el método `_que_bandeja_es` centraliza el acceso a las bandejas, asegurando que cualquier cambio en la estructura interna no afecte al usuario.

En conclusión, la separación entre la parte pública y privada en el TDA *Gestor de Correos* asegura robustez, modularidad y facilidad de uso, además de proteger la integridad de los datos internos del sistema.

TDA Dataframe

Francisco Carlos y Francisco Pellicer Alarcón

December 2024

1 Especificación del TDA DataFrame

1.1 Nombre y Descripción

Nombre del TDA: DataFrame

Descripción: El TDA describe una estructura de datos que simula una tabla con filas y columnas, donde cada columna puede contener diferentes tipos de datos. Proporciona operaciones para acceder a sublistas de filas y columnas, realizar cálculos estadísticos (como máximo y media), gestionar valores nulos y visualizar los datos. Además, maneja excepciones para casos de acceso a índices fuera de rango o columnas no existentes. Su objetivo es ofrecer una forma eficiente de manipular y consultar datos en una estructura tabular.

1.2 Dominio de los Datos

El TDA manipula un **DataFrame**, que es una estructura bidimensional similar a una tabla. Los tipos de datos utilizados en las funciones del TDA son los siguientes:

- **Reales** (R)
- **Cadenas de caracteres** (S)
- **Booleanos** (B)
- **Tupla** (T)
- **Array (Lista de elementos)** (A)
- **Valores nulos** (`None` o NaN):

1.3 Especificación de las Operaciones

A continuación, se describen las operaciones principales que el TDA DataFrame puede realizar, detallando sus precondiciones, lo que devuelven y las excepciones que pueden generar.

1. `insertar_fila(self, fila: Array, posicion=None) -> bool`

   - **Descripción**: Inserta una fila en una posición específica del DataFrame. Si no se especifica la posición, la fila se inserta al final.
   - **Precondiciones**:
     - `fila` debe ser un objeto de tipo `Array`.
     - El número de elementos en `fila` debe coincidir con el número de columnas del DataFrame.
   - **Retorna**:
     - `True` si la fila se inserta correctamente.
   - **Excepciones**:
     - *IndexError*: Si la posición está fuera de rango.
     - *TamanyoFilasColumnasError*: Si la fila no tiene el mismo número de elementos que las columnas.

2. `insertar_columna(self, nombre_columna: str, datos_columna: Array = None) -> bool`

   - **Descripción**: Inserta una nueva columna en el DataFrame. Si no se proporcionan datos de columna, se llenan con `None`. Si los datos no coinciden exactamente con el número de filas, se completan los datos faltantes con `None`.
   - **Precondiciones**:
     - `nombre_columna` debe ser un `str`.
     - `datos_columna` debe ser un objeto `Array`.
   - **Retorna**:
     - `True` si la columna se inserta correctamente.
   - **Excepciones**:
     - *TamanyoFilasColumnasError*: Si el número de elementos de la columna excede el número de filas.

3. `eliminar_columna(self, nombre_columna: str) -> bool`

   - **Descripción**: Elimina una columna en el DataFrame por su nombre.
   - **Precondiciones**:
     - `nombre_columna` debe ser un `str`.
   - **Retorna**:
     - `True` si la columna se elimina correctamente.
   - **Excepciones**:
     - *ColumnaNoExisteError*: Si no se encuentra la columna con el nombre dado.

4. `eliminar_fila(self, posicion: int) -> bool`

   - **Descripción**: Elimina una fila en el DataFrame en la posición especificada y desplaza las filas restantes hacia arriba.
   - **Precondiciones**:
     - `posicion` debe ser un índice válido dentro del rango de filas.
   - **Retorna**:
     - `True` si la fila se elimina correctamente.
   - **Excepciones**:
     - *IndexError*: Si la posición de la fila está fuera del rango permitido.

5. `columna_existe(self, nombre_columna: str) -> bool`

   - **Descripción**: Verifica si una columna existe en el DataFrame según su nombre.
   - **Precondiciones**:
     - `nombre_columna` debe ser un `str`.
   - **Retorna**:
     - `True` si la columna existe en el DataFrame.
     - `False` en caso contrario.
   - **Excepciones**: Ninguna

6. `head(n_filas: int) -> list[Array]`:

   - **Descripción**: Obtiene las primeras n filas del DataFrame.
   - **Precondiciones**: El parámetro n_filas debe ser un entero positivo.
   - **Valor de retorno**: Retorna un array con las primeras n_filas del DataFrame.
   - **Excepciones**:
     - *NumeroDeFilasExcedidoError*: Si el número de filas solicitado es mayor que el número de filas disponibles en el DataFrame.

7. `tail(n_filas: int) -> list[Array]`:

   - **Descripción**: Obtiene las últimas n filas del DataFrame.
   - **Precondiciones**: El parámetro n_filas debe ser un entero positivo.
   - **Retorna:**: Retorna un array con las últimas n_filas del DataFrame.
   - **Excepciones**:
     - *NumeroDeFilasExcedidoError*: Si el número de filas solicitado es mayor que el número de filas disponibles en el DataFrame.

8. `shape() -> tuple`:
   - **Descripción**: Obtiene la forma del DataFrame, es decir, el número de filas y columnas.
   - **Precondiciones**: No tiene precondiciones adicionales.
   - **Retorna:**: Retorna una tupla $(num_filas, num_columnas)$ que indica el número de filas y columnas en el DataFrame.
   - **Excepciones** : Ninguna
9. `max_columna(nombre_columna: str) -> float`:
   - **Descripción**: Obtiene el valor máximo de una columna específica del DataFrame, asegurándose de que todos los valores sean 'int', 'float' o 'None'.
   - **Precondiciones**: El parámetro *nombre_columna* debe ser una cadena que corresponda a una columna existente en el DataFrame.
   - **Retorna:**: Retorna el valor máximo en la columna especificada.
   - **Excepciones**:
     - *ColumnaNoExisteError*: Si la columna indicada no existe en el DataFrame.
     - *TipoDatoInvalidoError*: Si algún valor de la columna no es de tipo int, $float$ o $None$.
10. `mean_columna(nombre_columna: str) -> float`:
    - **Descripción**: Obtiene el valor medio (promedio) de los valores en una columna específica.
    - **Precondiciones**: El parámetro *nombre_columna* debe ser una cadena que corresponda a una columna existente en el DataFrame.
    - **Retorna:**: Retorna el valor promedio de la columna especificada.
    - **Excepciones**:
      - *ColumnaNoExisteError*: Si la columna indicada no existe en el DataFrame.
      - *ValoresInvalidosColumnaError*: Si la columna no tiene valores válidos (numéricos) para calcular la media.
11. `reemplazar_nulos(valor_reemplazo: object) -> None`:
    - **Descripción**: Reemplaza todos los valores nulos (None) del DataFrame con un valor proporcionado, si no los hay, no hace nada.
    - **Precondiciones**: El parámetro *valor_reemplazo* puede ser cualquier tipo de objeto que se utilizará para reemplazar los valores nulos.
    - **Retorna:**: No retorna ningún valor. Modifica el DataFrame directamente reemplazando los valores nulos por el valor proporcionado.

- **Excepciones**: Ninguna

12. `visualizar_filas_nulas() -> None`:
    - **Descripción**: Identifica y visualiza el conjunto de filas que contienen, al menos, una celda con valor nulo.
    - **Precondiciones**: No tiene precondiciones adicionales.
    - **Retorna:**: No retorna ningún valor. Imprime las filas que contienen al menos un valor nulo.
    - **Excepciones**: Ninguna
13. `visualizar_rango_filas(inicio: int, fin: int) -> None`:
    - **Descripción**: Muestra un rango de filas del DataFrame entre los índices especificados, con un formato alineado.
    - **Precondiciones**: Los parámetros *inicio* y fin deben ser índices válidos dentro del rango de filas del DataFrame, donde $inicio \leq fin$.
    - **Retorna:**: No retorna ningún valor. Imprime las filas dentro del rango especificado.
    - **Excepciones**:
      - *IndexError*: Si los índices proporcionados son inválidos (fuera del rango de filas).
14. `visualizar_fila(indice: int) -> None`:
    - **Descripción**: Muestra el contenido de una fila específica del DataFrame, con formato alineado.
    - **Precondiciones**: El parámetro *indice* debe ser un número entero que represente el índice de la fila a visualizar, y debe estar dentro del rango de filas.
    - **Retorna:**: No retorna ningún valor. Imprime el contenido de la fila especificada.
    - **Excepciones**:
      - *IndexError*: Si el índice proporcionado está fuera del rango de filas.
15. `visualizar_columna(nombre_columna: str) -> None`:
    - **Descripción**: Muestra los valores de una columna específica del DataFrame.
    - **Precondiciones**: El parámetro *nombre_columna* debe ser una cadena que corresponda a una columna existente en el DataFrame.
    - **Retorna:**: No retorna ningún valor. Imprime los valores de la columna especificada.
    - **Excepciones**:
      - *ColumnaNoExisteError*: Si la columna indicada no existe en el DataFrame.

2 Representación

```
struct DataFrame {
    Array columnas;             // Array que contiene las columnas del DataFrame
    int capacidad;              // Capacidad máxima del DataFrame (número de filas)
    Array dataframe[][];        // Matriz bidimensional (array de arrays) para los datos
    int num_filas;              // Número de filas actuales en el DataFrame


struct Array {
    int capacidad;              // Capacidad inicial del array
    int longitud;               // Longitud actual del array
    object datos[];             // Datos almacenados en el array
```

3 Implementación

El **TDA Array** es la estructura base que utiliza el **DataFrame** para almacenar las filas y columnas de manera eficiente. Cada fila del **DataFrame** es representada como un **Array**, donde se almacenan los valores correspondientes a cada columna. Este enfoque modular facilita tanto el acceso como la manipulación de los datos.

3.1 Ventajas:

- **Acceso rápido a los elementos:** Los **Arrays** proporcionan un acceso eficiente a los valores mediante índices, lo que es crucial cuando se trabaja con grandes volúmenes de datos. Esto permite realizar operaciones de manera rápida y directa.
- **Modularidad:** Al manejar las filas y columnas de manera separada, el código es más organizado y flexible. Esto permite realizar cambios internos sin afectar la interfaz pública del **DataFrame**.

3.2 Separación entre Parte Pública y Privada

Una clara separación entre las funciones públicas y privadas se ha mantenido en el diseño, lo cual es esencial para:

- **Encapsulamiento:** Las funciones privadas, como `_get_columnas`, `_get_dataframe`, y `_get_num_filas`, gestionan el acceso a los datos internamente. Al ser privadas, limitan la manipulación directa de la estructura interna.
- **Flexibilidad y mantenimiento:** Al separar la lógica interna de la interfaz pública, el código se vuelve más fácil de mantener y ampliar sin afectar

la experiencia del usuario. Esto permite realizar modificaciones internas sin comprometer las funcionalidades públicas.

3.3 Parte Pública del TDA

La parte pública del **DataFrame** ofrece métodos que permiten interactuar con los datos de forma directa y sencilla. Algunas de las operaciones más relevantes son:

- **Métodos estadísticos:**
  - `max_columna`: Calcula el valor máximo de una columna específica.
  - `mean_columna`: Calcula el valor medio de una columna específica.
- **Métodos de manipulación y visualización:**
  - `reemplazar_nulos`: Reemplaza los valores nulos por un valor dado.
  - `visualizar_filas_nulas`: Muestra las filas que contienen al menos un valor nulo.
  - `visualizar_rango_filas`: Muestra un rango específico de filas.
  - `visualizar_columna`: Muestra los valores de una columna específica.

3.4 Métodos Privados

Estos métodos no son accesibles desde fuera de la clase y se utilizan exclusivamente para manejar los datos internamente:

- `_actualizar_num_filas(self, cantidad_filas: int) -> None`: Actualiza el número total de filas en el **DataFrame**. Es utilizado para ajustar dinámicamente el tamaño del DataFrame.
- `_get_num_filas(self) -> int`: Retorna el número total de filas en el **DataFrame**. Facilita el control sobre el tamaño de la estructura interna.
- `_get_capacidad(self) -> int`: Retorna la capacidad actual del **DataFrame**. Permite gestionar cuántas filas puede almacenar antes de redimensionar.
- `_get_columnas(self) -> Array[]`: Retorna las columnas del **DataFrame**. Se usa para acceder a las columnas para operaciones internas como estadísticas.
- `_get_dataframe(self) -> Array[][]`: Retorna las filas del **DataFrame**. Facilita el acceso a los datos completos para manipulaciones internas.
- `_redimensionar(self) -> None`: Redimensiona el **DataFrame** duplicando su capacidad. Asegura espacio suficiente cuando se superan los límites actuales.

3.5 Decisiones de Diseño y Organización del Código

El diseño y organización del **DataFrame** se basa en decisiones clave que aseguran su eficiencia y escalabilidad:

- **Eficiencia:** El uso del **Array** como estructura subyacente optimiza el acceso y la manipulación de los datos.
- **Modularidad:** El código está organizado en métodos que cumplen funciones específicas, lo que facilita el mantenimiento y la reutilización del código. Cada método realiza una tarea concreta lo que mejora la comprensión y el control sobre el código.
- **Manejo de errores y validaciones:** Se han implementado verificaciones para asegurar que los datos sean válidos antes de realizar operaciones. Por ejemplo, se valida que los valores sean numéricos antes de calcular estadísticas y se manejan excepciones si el usuario intenta acceder a columnas no existentes o a datos no válidos.
- **Flexibilidad y escalabilidad:** El diseño modular permite que el **DataFrame** evolucione fácilmente. Si se necesitan nuevas funcionalidades o cambios en la estructura interna, el código puede adaptarse sin afectar la interfaz pública ni las funcionalidades existentes.

3.6 Ventajas del Uso del TDA Array dentro del DataFrame

El uso del **TDA Array** como base del **DataFrame** aporta varias ventajas:

- **Simplificación de la manipulación de datos:** Los **Arrays** facilitan la manipulación de los datos dentro de las filas y columnas, simplificando las operaciones y mejorando la claridad del código.
- **Mejor desempeño:** El acceso directo a los elementos de las filas y columnas dentro del **Array** mejora el rendimiento, especialmente en operaciones frecuentes como la visualización de datos o el cálculo de estadísticas.
- **Claridad y organización:** Separar las filas y columnas en objetos **Array** hace que el diseño sea más limpio y fácil de comprender. Cada operación sobre el **DataFrame** puede conceptualizarse como una manipulación de estos **Arrays**, lo que facilita el mantenimiento y la integración de nuevas funcionalidades.

4 Main.py

El fichero `main.py` permite probar y ejecutar las funcionalidades requeridas con los tres TDAs. Para ello, se deben importar los otros ficheros que contienen las clases `Correo`, `GestorCorreo` y `DataFrame`. A lo largo de todo el archivo, se van probando todos los métodos públicos que se "ofrecen" al usuario, imprimiendo por pantalla los resultados de estos y realizando un pequeño control de los errores que se consideran más comunes.

Cierto es que los tres TDAs se prueban por separado, sin embargo, en el caso de `Correo` y `GestorCorreo`, estas clases se nutren entre sí. Es decir, `GestorCorreo` no tendría sentido sin la clase `Correo`, ya que sin ella no tendría objetos con los que operar. De la misma manera, la clase `Correo` necesita del gestor para que exista un flujo de correos y su transmisión. Cabe añadir que ciertos métodos de la clase `Correo` han sido reutilizados en las funcionalidades de `GestorCorreo`, como el método `buscar_palabra`. El tercer TDA, `DataFrame`, no tiene relación alguna con los anteriores.

En cuanto a las ventajas de importar los módulos, contamos con una distribución de las funcionalidades en módulos independientes. El fichero `main.py` se centra en cómo usar los métodos, en lugar de en cómo están implementados. Además, cada módulo encapsula una responsabilidad específica, asegurándose de que los detalles internos de cada TDA estén protegidos. Nos garantiza, también, una delegación correcta de las responsabilidades de cada clase, lo que permite reutilizar código y asegura que el sistema pueda ser mantenido a largo plazo.